

CS 378: Automated Planning for Robots - HW 2

Understanding the Limits of Classical Heuristics

Report

This report presents three families of planning problems that demonstrate when classical planning heuristics are accurate (Q1) or very inaccurate (Q2, Q4). All measurements were obtained by running Fast Downward with eager_greedy search.

Accuracy is measured as the ratio $r = h(s_0) / \text{plan_cost}$, where $h(s_0)$ is the initial heuristic value reported by Fast Downward and plan_cost is the cost of the plan found by greedy search. A ratio of 1 means the heuristic is perfectly accurate; a decreasing ratio indicates a very inaccurate heuristic.

Questions answered: Q1 (FF accurate), Q2 (FF inaccurate), Q4 (CG inaccurate, extra credit).

Q1 - Relaxed-Plan Heuristic (FF) is Accurate

Domain: Independent Tasks

Each problem instance contains n tasks. A single 'do-task(?t)' action completes any pending task with no side effects. Tasks share no resources and have no negative interactions with each other.

Why FF is accurate:

In the relaxed planning graph (delete effects ignored) every goal 'completed(ti)' is directly achievable by 'do-task(ti)', whose only precondition 'pending(ti)' is satisfied in the initial state. Because there are no shared or conflicting resources, the relaxed plan contains exactly n actions - exactly the same as the optimal real plan. $h^{ff} = h^* = n$, ratio = 1.00 for every instance.

Problem	Size (n)	h^{ff} (FF)	Plan cost	Ratio
p1	1	1	1	1.00
p2	2	2	2	1.00
p3	3	3	3	1.00
p4	4	4	4	1.00
p5	5	5	5	1.00

CS 378: Automated Planning for Robots - HW 2

Understanding the Limits of Classical Heuristics

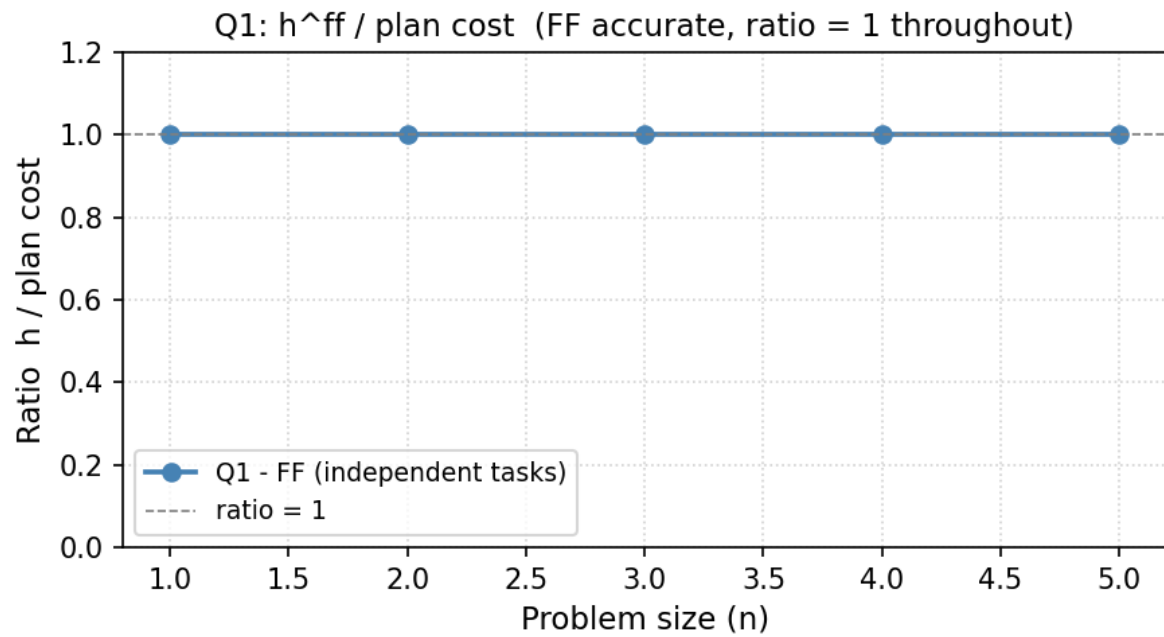


Figure 1 - Q1 ratio remains 1.00 for all problem sizes.

CS 378: Automated Planning for Robots - HW 2

Understanding the Limits of Classical Heuristics

Q2 - Relaxed-Plan Heuristic (FF) is Inaccurate

Domain: Key-Doors

There is ONE shared key and n doors, all initially closed. The goal is to open all doors. Actions:

- use-key(?d): pre: key-ready, door-closed(?d); effect: door-open(?d), key-used, NOT key-ready.
- renew-key: pre: key-used; effect: key-ready, NOT key-used.

Opening a door CONSUMES the key (sets key-used, removes key-ready). The key must be renewed before the next door can be opened. FF ignores this delete effect.

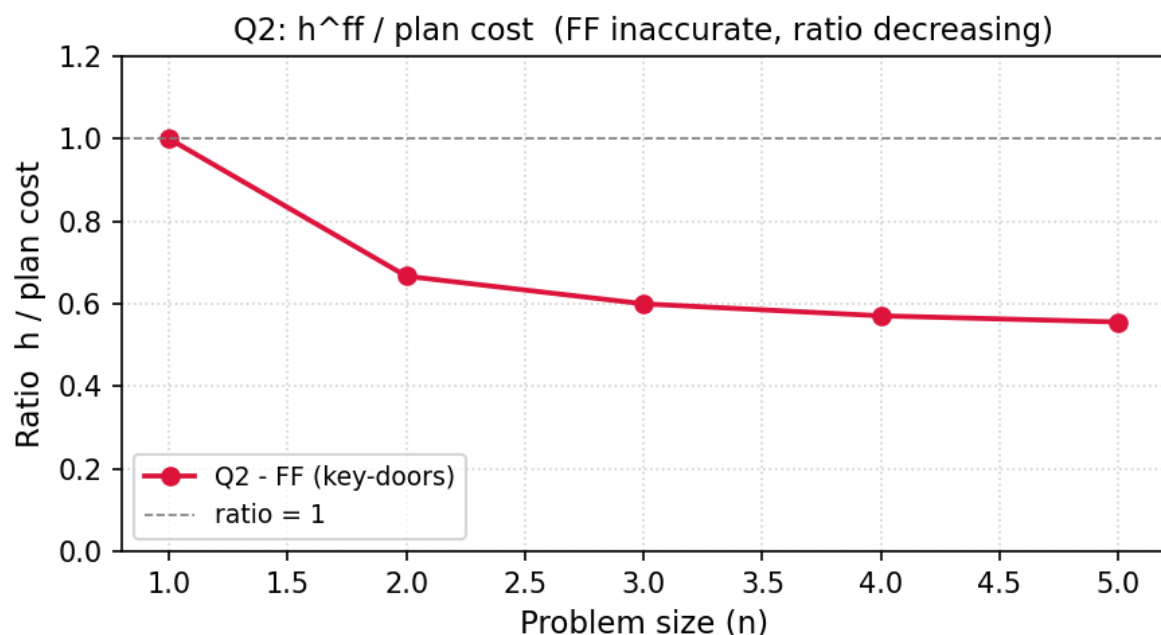
Why FF is inaccurate:

FF ignores the delete effect NOT key-ready. In the relaxed plan, key-ready is never removed, so all n doors can be opened by n consecutive use-key actions without any renewal. $h^{ff} = n$.

In the real problem after opening door d_1 the key is gone. The optimal plan must alternate: use-key, renew, use-key, renew, ..., use-key, giving $h^* = 2n - 1$.

Ratio = $n / (2n-1)$ decreases from 1.00 toward 0.50 as n grows.

Problem	Size (n)	h^{ff} (FF)	Plan cost	Ratio
p1	1	1	1	1.000
p2	2	2	3	0.667
p3	3	3	5	0.600
p4	4	4	7	0.571
p5	5	5	9	0.556



CS 378: Automated Planning for Robots - HW 2

Understanding the Limits of Classical Heuristics

Figure 2 - Q2 ratio decreases from 1.00 toward 0.50 as n grows.

CS 378: Automated Planning for Robots - HW 2

Understanding the Limits of Classical Heuristics

Q4 (Extra Credit) - Causal-Graph Heuristic (CG) is Inaccurate

Domain: Single-Gripper (one-gripper variant of the IPC Gripper benchmark)

A robot with ONE gripper must transport n balls from room A to room B, one ball at a time. Problem sizes: $n = 2, 4, 6, 8, 10$ balls. Actions: `move(from,to)`, `pick(ball,room)`, `drop(ball,room)`.

Optimal cost:

Each trip carries 1 ball. Forward trip: `pick` + `move(A->B)` + `drop` = 3. Return trip: `move(B->A)` = 1. For n balls: $h^* = 3n + (n-1) = 4n - 1$ (last trip has no return).

Why CG is inaccurate:

The CG heuristic processes each ball's Domain Transition Graph (DTG) independently. For each ball it sees a cost of 3 (`pick` + shared `move` + `drop`) but treats the single `A->B` move as shared across all balls. It does not account for the return trip the robot must make between deliveries. The estimate is $h_{CG} = 3n$.

Ratio = $3n / (4n - 1)$ decreases toward 0.75 as n grows.

Problem	Balls (n)	h_{CG} (CG)	Plan cost	Ratio
p1	2	6	7	0.857
p2	4	12	15	0.800
p3	6	18	23	0.783
p4	8	24	31	0.774
p5	10	30	39	0.769

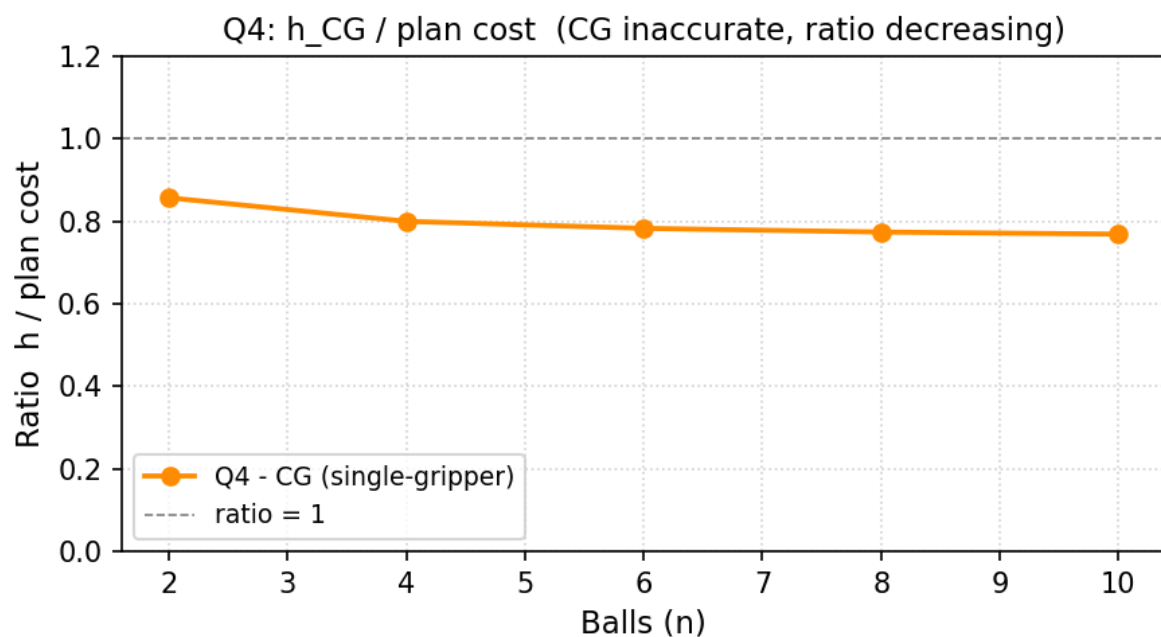


Figure 3 - Q4 ratio decreases as more balls require more return trips.

CS 378: Automated Planning for Robots - HW 2

Understanding the Limits of Classical Heuristics

Summary

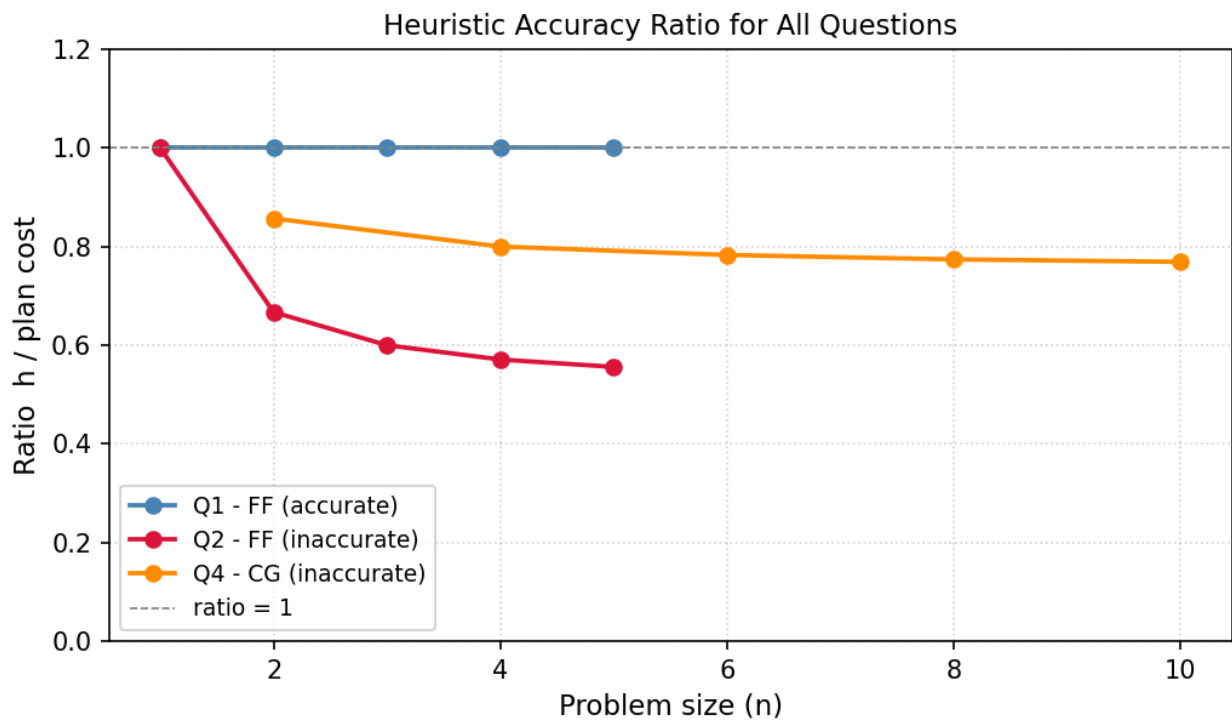


Figure 4 - Q1 stays at 1.0; Q2 and Q4 decrease with problem size.

Summary of findings:

Q1 (Independent Tasks): FF is perfectly accurate (ratio = 1.00) because every goal can be achieved by a single independent action. No delete effects matter.

Q2 (Key-Doors): FF is very inaccurate. It ignores that opening a door consumes the key, so the relaxed plan opens all n doors in n steps ($h^{ff} = n$) while the real plan needs $2n-1$ steps. The ratio $n/(2n-1)$ decreases from 1.00 toward 0.50.

Q4 (Single-Gripper, extra credit): CG is inaccurate. It sums the per-ball DTG costs without accounting for the return trips the robot must make between deliveries. $h_{CG} = 3n$ while plan cost = $4n-1$. The ratio decreases monotonically from ~ 0.857 toward 0.75.